

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(МОСКОВСКИЙ ПОЛИТЕХ)
Факультет информационных технологий
Кафедра «Информационная безопасность»

Информационная система для ведения заказов компанией
Пояснительная записка
К курсовому проекту по дисциплине «Проектирование и
администрирование баз данных»
Вариант 4. Информационная система для ведения заказов компанией

Выполнил студент:
группы 251-951
Хакимов Дени Равилевич

Руководитель проекта:
к.т.н., доцент
Харченко Елена Алексеевна

Москва, 2026 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	3
1 ПОСТАНОВКА ЗАДАЧИ.....	5
1.1 Цель работы.....	5
1.2 Задачи работы.....	5
1.3 Ход работы.....	6
2 ПРОЕКТИРОВАНИЕ И РАЗРАБОТКА.....	8
2.1 Описание функциональных возможностей приложения.....	8
2.2 Описание и схема базы данных.....	11
2.3 Структура кода.....	20
3 ОСНОВНЫЕ СЦЕНАРИИ РАБОТЫ.....	24
3.1 Сценарий №1 – Регистрация пользователя и добавление данных о заказчике	24
3.2 Сценарий №2 – Оформление заказа пользователем	24
3.3 Сценарий №3 – Создание нового пользователя и управление справочниками администратором	25
3.4 Сценарий №4 – Просмотр истории всех заказов и финансовая аналитика.....	26
ЗАКЛЮЧЕНИЕ	27
СПИСОК ЛИТЕРАТУРЫ И ИНТЕРНЕТ-РЕСУРСЫ.....	29
ПРИЛОЖЕНИЕ.....	30

ВВЕДЕНИЕ

В настоящее время цифровая трансформация и автоматизация ключевых бизнес-процессов являются фундаментальными факторами успешной коммерческой деятельности предприятий на динамично развивающемся рынке товаров и услуг. Эффективная организация оптовой и розничной торговли требует внедрения комплексных программных решений, обеспечивающих централизованный учет контрагентов, оперативное оформление клиентских сделок, надежное хранение номенклатуры продукции и мониторинг финансово-экономических показателей.

Существенная проблематика в сфере оптовой и мелкооптовой коммерции связана с обработкой больших массивов гетерогенной информации, поддержанием строгой ссылочной целостности, непротиворечивости и хронологической актуальности данных на всех этапах жизненного цикла заказа. Особую сложность представляет логистический аспект: товары одного предприятия могут доставляться контрагентам различными способами (курьерской службой, автомобильным транспортом, транспортными компаниями или путем самовывоза со складов), причем для каждой товарной позиции предопределен собственный перечень допустимых способов доставки с индивидуальной тарифной сеткой. Недостаточный учет указанных факторов в ручном или полуавтоматическом режиме неизбежно приводит к тарифным коллизиям, задержкам отгрузки и прямым финансовым издержкам компании.

С точки зрения архитектуры программных систем критически важным требованием выступает математически выверенное проектирование схемы реляционной базы данных. Пренебрежение принципами нормализации на этапе концептуального и логического моделирования влечет за собой возникновение опасных аномалий включения (insertion anomalies), обновления (update anomalies) и удаления (deletion anomalies). В условиях многопользовательского доступа указанные дефекты способны вызывать

десинхронизацию истории сделок и утрату критически важных финансовых сведений.

Настоящий курсовой проект посвящен комплексному проектированию и программной разработке клиент-серверной информационной системы для ведения заказов торговой компанией (Вариант 4). В работе детально исследован полный цикл создания программного продукта: от анализа предметной области, построения инфологической модели Чена и приведения реляционной схемы к нормальной форме Бойса-Кодда (НФБК) до программной реализации сетевого ТСР-сервера с многопоточным пулом обработки запросов и полнофункционального клиентского интерфейса на базе фреймворка Qt6 с разграничением прав доступа на основе ролевой модели.

1 ПОСТАНОВКА ЗАДАЧИ

1.1 Цель работы

Целью курсового проекта является проектирование и программная реализация защищенной высокопроизводительной клиент-серверной информационной системы, предназначенной для комплексной автоматизации процессов ведения заказов торговой компанией, строгого учета predetermined способов и тарифов доставки, надежного хранения сведений о юридических лицах-контрагентах, а также проведения финансово-аналитического мониторинга коммерческой деятельности.

1.2 Задачи работы

Для достижения поставленной цели в процессе выполнения курсового проекта решены следующие ключевые задачи:

1. Проведение детального анализа предметной области оптовой торговли с учетом predetermined способов доставки и формализация бизнес-требований компании (Вариант 4).
2. Построение концептуальной инфологической модели данных (ER-диаграммы Чена) с выявлением сущностей, их атрибутивного состава и кардинальности связей.
3. Проектирование первичной и итоговой реляционных моделей базы данных с раскрытием связей «многие-ко-многим» и оптимизацией отношений со связностью «один-к-одному».
4. Проведение строгого математического анализа функциональных зависимостей и доказательство соответствия реляционной схемы нормальной форме Бойса-Кодда (НФБК).
5. Разработка физической модели базы данных на диалекте SQL для СУБД PostgreSQL с внедрением ограничений целостности (PRIMARY KEY, FOREIGN KEY, UNIQUE, CHECK) и каскадных политик.

6. Проектирование бинарно-текстового протокола сетевого взаимодействия поверх транспортного протокола TCP на базе сериализации структурированных сообщений формата JSON.

7. Разработка серверного компонента на языке C++ с применением библиотеки Qt6 Network (класс QTcpServer), паттерна «Одиночка» (Singleton) для управления соединениями с СУБД и криптографического хэширования паролей алгоритмом SHA-256.

8. Создание кроссплатформенного графического клиентского приложения на базе Qt6 Widgets с ролевым разделением интерфейса (Администратор, Заказчик), модулем формирования заказов и встроенным блоком финансово-экономической статистики.

9. Проведение комплексного тестирования основных пользовательских и административных сценариев функционирования системы.

1.3 Ход работы

Разработка информационной системы осуществлялась поэтапно в соответствии с каноническими принципами программной инженерии:

- **Этап системного анализа:** На первом этапе произведена декомпозиция бизнес-логики компании. Выявлены информационные потоки между заказчиками (юридическими лицами), менеджерами и администраторами системы. Определены требования к номенклатуре товаров, способам доставки и расчету затрат.

- **Этап инфологического моделирования:** Спроектирована концептуальная ER-диаграмма, фиксирующая семантику предметной области без привязки к конкретной целевой СУБД.

- **Этап логического проектирования:** Сущности и ассоциации инфологической модели преобразованы в набор нормализованных реляционных отношений. Проведено соединение отношений «Пользователи» и «Заказчики» для устранения избыточных промежуточных связей.

- **Этап теоретической нормализации:** Составлено минимальное неприводимое покрытие функциональных зависимостей. Доказано отсутствие транзитивных и частичных зависимостей от ключей, подтверждено соответствие схемы критерию НФБК.

- **Этап физического проектирования:** Написан DDL-скрипт развертывания схемы базы данных в PostgreSQL (с резервной поддержкой встраиваемой СУБД SQLite для автономного развертывания). Реализованы внешние ключи с режимами ON DELETE CASCADE и ON DELETE RESTRICT.

- **Этап программной реализации:** Разработана архитектура TCP-клиент-сервера. На стороне сервера создан пул ClientHandler и модуль DatabaseManager (Singleton). На стороне клиента реализован класс NetworkClient и графические модули MainWindow, ProductView, OrderView, DeliveryView, CustomerView.

- **Этап тестирования и верификации:** Проведено верификационное тестирование сценариев авторизации, редактирования профилей, оформления заказов с калькуляцией доставки и формирования финансового отчета.

2 ПРОЕКТИРОВАНИЕ И РАЗРАБОТКА

2.1 Описание функциональных возможностей приложения

Разработанный программный комплекс представляет собой клиент-серверную систему, реализующую полный цикл взаимодействия торговой компании со своими контрагентами. Архитектура системы построена на строгом разграничении прав доступа на основе ролевой модели (Role-Based Access Control, RBAC). В системе выделяются три ключевые категории пользователей: неавторизованные пользователи, заказчики (клиенты компании) и системные администраторы.

Наглядное представление функциональных возможностей приложения и распределения прецедентов использования между действующими лицами (актерами) приведено на рисунке 1.

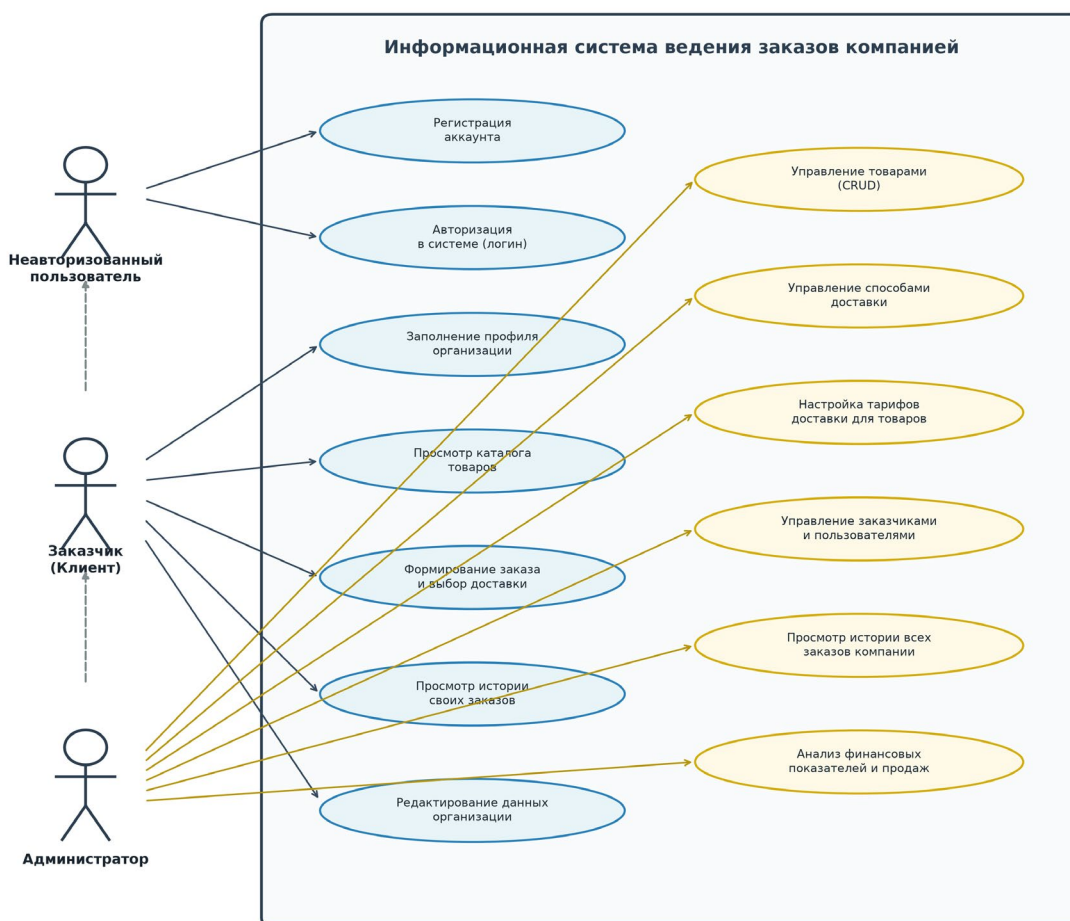


Рисунок 1 – UML-диаграмма вариантов использования системы

2.1.1 Система авторизации и регистрации

Модуль аутентификации и регистрации обеспечивает безопасный вход в систему и первичное заведение учетных записей. В целях обеспечения информационной безопасности в системе реализованы следующие механизмы:

- **Валидация пароля:** Проверка сложности пароля при регистрации с использованием регулярных выражений: длина пароля должна составлять не менее 8 символов, содержать прописные и строчные буквы латинского алфавита, цифры и специальные символы.

- **Криптографическая защита:** Пароли пользователей ни при каких обстоятельствах не сохраняются в открытом виде. На стороне сервера вычисляется криптографический хэш по алгоритму SHA-256 с использованием класса `QCryptographicHash`. В базе данных сохраняется исключительно 64-символьная шестнадцатеричная строка хэша.

- **Ролевое разграничение:** Каждому пользователю сопоставляется роль из таблицы `roles`. При первичной самостоятельной регистрации пользователю автоматически присваивается базовая клиентская роль (`customer/user`). Роль администратора (`admin`) предоставляет расширенные привилегии управления конфигурацией системы.

2.1.2 Пользователь (Заказчик)

Пользователь с ролью «Заказчик» представляет юридическое лицо, приобретающее продукцию компании. Для данной роли доступны следующие функциональные возможности:

- **Каталог продукции:** Просмотр актуальной номенклатуры товаров с отображением базовой цены за единицу, подробного технического описания (справочной информации) и признака доступности доставки.

- **Формирование заказа:** Возможность добавления выбранных товаров в заказ с указанием необходимого количества и выбором конкретного способа доставки из списка предварительно определенных тарифов для данного товара.

- **История заказов:** Отображение персональной истории совершенных покупок с возможностью интерактивного раскрытия состава каждого заказа, отображением выбранных способов доставки, начисленной стоимости транспортировки и итоговой суммы сделки.

- **Управление профилем:** Ввод и актуализация реквизитов юридического лица: официального наименования организации, ФИО ответственного контактного лица, юридического/фактического адреса доставки и номера телефона.

2.1.3 Администратор

Роль «Администратор» наделена полномочиями полного контроля за нормативно-справочной информацией и мониторингом торговой деятельности:

- **Управление товарами:** Добавление новых товарных позиций, корректировка цен и справочных данных, а также удаление позиций. Удаление товаров защищено ссылочной целостностью ON DELETE RESTRICT: товар невозможно удалить, если он уже зафиксирован в архивных сделках.

- **Управление логистикой:** Ведение справочника способов доставки с указанием регламентных сроков (скорости доставки) и настройка матрицы тарифов (delivery_tariffs), связывающей товары с доступными способами и фиксирующей базовые логистические ставки.

- **Управление заказчиками:** Просмотр реестра всех зарегистрированных контрагентов с контактными данными и возможностью удаления учетных записей. При удалении заказчика каскадно очищаются связанные заказы в рамках регламента обслуживания БД.

- **Аудит сделок:** Глобальный доступ ко всем заказам, оформленным всеми контрагентами компании, с детализацией по позициям.

2.1.4 Модуль доставки и финансовой аналитики

В соответствии с техническим заданием (Вариант 4), информационная система обязана отслеживать финансовую сторону деятельности

предприятия. В клиентском приложении реализован специализированный аналитический модуль, агрегирующий данные из базы данных:

- **Объем заказов:** Суммарное количество успешно проведенных заказов в системе.
- **Товарооборот:** Совокупное число реализованных товарных единиц (штук) по всем закрытым позициям.
- **Выручка от товаров:** Общая стоимость реализованной продукции (без учета логистических надбавок).
- **Затраты на доставку:** Суммарный объем финансовых средств, начисленных за оказание транспортных и курьерских услуг.
- **Средний чек (AOV):** Средневзвешенная стоимость одной комплексной сделки с учетом стоимости транспортировки.
- **Аналитика номенклатуры:** Интерактивная таблица продаж по номенклатуре: для каждого наименования вычисляется общее количество проданных штук и суммарный финансовый объем выручки, что позволяет моментально выявлять наиболее маржинальные и востребованные позиции ассортимента.

2.2 Описание и схема базы данных

2.2.1 Структура базы данных

База данных информационной системы функционирует под управлением реляционной СУБД PostgreSQL (с поддержкой резервного переключения на SQLite в автономном режиме). Логическая структура спроектирована в виде 8 взаимосвязанных нормализованных отношений:

- **1. roles (Роли)** – Справочник системных ролей пользователей. Содержит суррогатный первичный ключ `role_id` (SERIAL) и уникальное наименование роли `role_name` (VARCHAR(100), UNIQUE), например: 'admin', 'customer', 'user'.
- **2. users (Пользователи)** – Учетные записи системы. Включает `user_id` (SERIAL PRIMARY KEY), внешний ключ `role_id` (REFERENCES roles ON

DELETE RESTRICT), уникальный электронный адрес email (VARCHAR(255), UNIQUE) и хэш пароля password_hash (VARCHAR(255)), сформированный алгоритмом SHA-256.

- **3. customers (Заказчики)** – Реквизиты юридических лиц-контрагентов. Отношение связано связью «один-к-одному» с таблицей users через user_id (UNIQUE, ON DELETE CASCADE). Включает customer_id (PK), organization_name (VARCHAR(255)), contact_person (VARCHAR(255)), address (TEXT) и phone (VARCHAR(20), UNIQUE).

- **4. orders (Заказы)** – Реестр совершенных торговых сделок. Содержит order_id (SERIAL PRIMARY KEY), внешний ключ customer_id (REFERENCES customers ON DELETE CASCADE) и дату совершения покупки purchase_date (DATE DEFAULT CURRENT_DATE).

- **5. products (Товары)** – Каталог продаваемой продукции компании. Включает product_id (SERIAL PRIMARY KEY), наименование product_name (VARCHAR(255)), цену единицы price (NUMERIC(12,2) CHECK (price >= 0)), развернутую справочную информацию reference_info (TEXT) и логический флаг доступности доставки is_delivery_available (BOOLEAN DEFAULT TRUE).

- **6. delivery_methods (Способы доставки)** – Справочник доступных способов транспортировки товаров. Содержит delivery_method_id (SERIAL PRIMARY KEY), уникальное наименование method_name (VARCHAR(150), UNIQUE) и регламентную скорость доставки speed (VARCHAR(100), например: '1-2 дня', 'В день заказа').

- **7. delivery_tariffs (Тарифы доставки)** – Таблица связки «многие-ко-многим» между товарами и способами доставки. Фиксирует допустимые predetermined способы транспортировки для каждого конкретного товара и устанавливает базовую ставку: delivery_tariff_id (PK), product_id (FK), delivery_method_id (FK) и base_delivery_price (NUMERIC(12,2) CHECK (base_delivery_price >= 0)). Введена уникальность пары (product_id, delivery_method_id).

- **8. order_items (Позиции заказа)** – Товарные строки, входящие в конкретную сделку. Содержит order_item_id (SERIAL PRIMARY KEY), order_id (FK CASCADE), product_id (FK RESTRICT), delivery_method_id (FK RESTRICT), фактическую зафиксированную стоимость доставки actual_delivery_cost (NUMERIC(12,2) CHECK (actual_delivery_cost >= 0)) и количество единиц quantity (INT CHECK (quantity > 0)). Использование стратегии RESTRICT гарантирует неизменность финансовых отчетов при изменении текущих тарифов.

2.2.2 Обеспечение целостности данных

В реляционной модели базы данных реализована многоуровневая система обеспечения целостности информации:

- **Сущностная целостность:** Гарантируется использованием автоинкрементных суррогатных первичных ключей (PRIMARY KEY типа SERIAL) во всех таблицах, а также бизнес-ограничениями UNIQUE для полей roles.role_name, users.email, customers.phone, customers.user_id, delivery_methods.method_name и составного ключа delivery_tariffs(product_id, delivery_method_id).

- **Ссылочная целостность:** Обеспечивается механизмом внешних ключей (FOREIGN KEY). Для связей слабой сущности заказа и позиций применен каскадный режим (ON DELETE CASCADE), гарантирующий автоматическую очистку зависимых данных. Для справочников товаров и способов доставки в позициях заказа применен режим ON DELETE RESTRICT ON UPDATE CASCADE, блокирующий физическое удаление сущностей, на которые ссылаются проведенные финансовые документы.

- **Доменная целостность:** Реализована на уровне ядра СУБД с помощью декларативных ограничений CHECK: цены товаров, базовые тарифы и фактические затраты на доставку ограничены условием неотрицательности (≥ 0), а объем заказываемой продукции строго положительным значением ($\text{quantity} > 0$).

2.2.3 Нормализация базы данных

Для подтверждения теоретической корректности спроектированной базы данных проведен строгий математический анализ нормализации отношений.

1. Первая нормальная форма (1НФ): Все атрибуты отношений являются атомарными (неделимыми), а кортежи таблиц не содержат повторяющихся групп или массивов. Каждый атрибут хранит скалярное значение соответствующего домена (целые числа, строки фиксированной длины, даты, вещественные числа).

2. Вторая нормальная форма (2НФ): Схема находится в 1НФ, и каждый неключевой атрибут функционально полно зависит от первичного ключа соответствующего отношения. В таблицах с составными альтернативными ключами (например, `delivery_tariffs`) базовый тариф зависит исключительно от полной пары (`product_id`, `delivery_method_id`), а не от отдельных ее частей.

3. Третья нормальная форма (3НФ): Схема находится во 2НФ, и в отношениях отсутствуют транзитивные функциональные зависимости неключевых атрибутов от первичного ключа. В таблице `order_items` фактическая стоимость доставки `actual_delivery_cost` фиксируется непосредственно для строки заказа и не вычисляется динамически через цепочку сторонних таблиц, что предотвращает аномалии при изменении справочников.

4. Нормальная форма Бойса-Кодда (НФБК): Отношение R находится в нормальной форме Бойса-Кодда тогда и только тогда, когда для любой нетривиальной функциональной зависимости $X \rightarrow Y$ левая часть (детерминант X) является суперключом отношения. Для схемы базы данных проекта получено следующее минимальное неприводимое множество функциональных зависимостей (ФЗ):

- roles: $\{ \text{role_id} \} \rightarrow \{ \text{role_name} \}; \{ \text{role_name} \} \rightarrow \{ \text{role_id} \}$
- users: $\{ \text{user_id} \} \rightarrow \{ \text{role_id}, \text{email}, \text{password_hash} \}; \{ \text{email} \} \rightarrow \{ \text{user_id}, \text{role_id}, \text{password_hash} \}$

- customers: { customer_id } → { user_id, organization_name, contact_person, address, phone }; { user_id } → { customer_id, ... }; { phone } → { customer_id, ... }
- orders: { order_id } → { customer_id, purchase_date }
- products: { product_id } → { product_name, price, reference_info, is_delivery_available }
- delivery_methods: { delivery_method_id } → { method_name, speed }; { method_name } → { delivery_method_id, speed }
- delivery_tariffs: { delivery_tariff_id } → { product_id, delivery_method_id, base_delivery_price }; { product_id, delivery_method_id } → { delivery_tariff_id, base_delivery_price }
- order_items: { order_item_id } → { order_id, product_id, delivery_method_id, actual_delivery_cost, quantity }

Поскольку во всех приведенных функциональных зависимостях каждый детерминант представляет собой потенциальный ключ (кандидат в ключи) соответствующего отношения, доказано, что реляционная схема базы данных находится в нормальной форме Бойса-Кодда (НФБК). Это гарантирует полное отсутствие аномалий избыточности при вставке, модификации и удалении записей.

2.2.4 Модели базы данных

В процессе проектирования базы данных были разработаны и сопоставлены четыре уровня моделей данных, отражающие последовательную трансформацию требований технического задания в программный код:

1. Инфологическая (концептуальная) ER-модель Чена представлена на рисунке 2. На схеме выделены ключевые сущности предметной области (Роль, Пользователь, Заказчик, Заказ, Позиция заказа, Товар, Способ доставки, Тариф доставки), их смысловые атрибуты и семантические типы связей (1:1, 1:M, M:M).

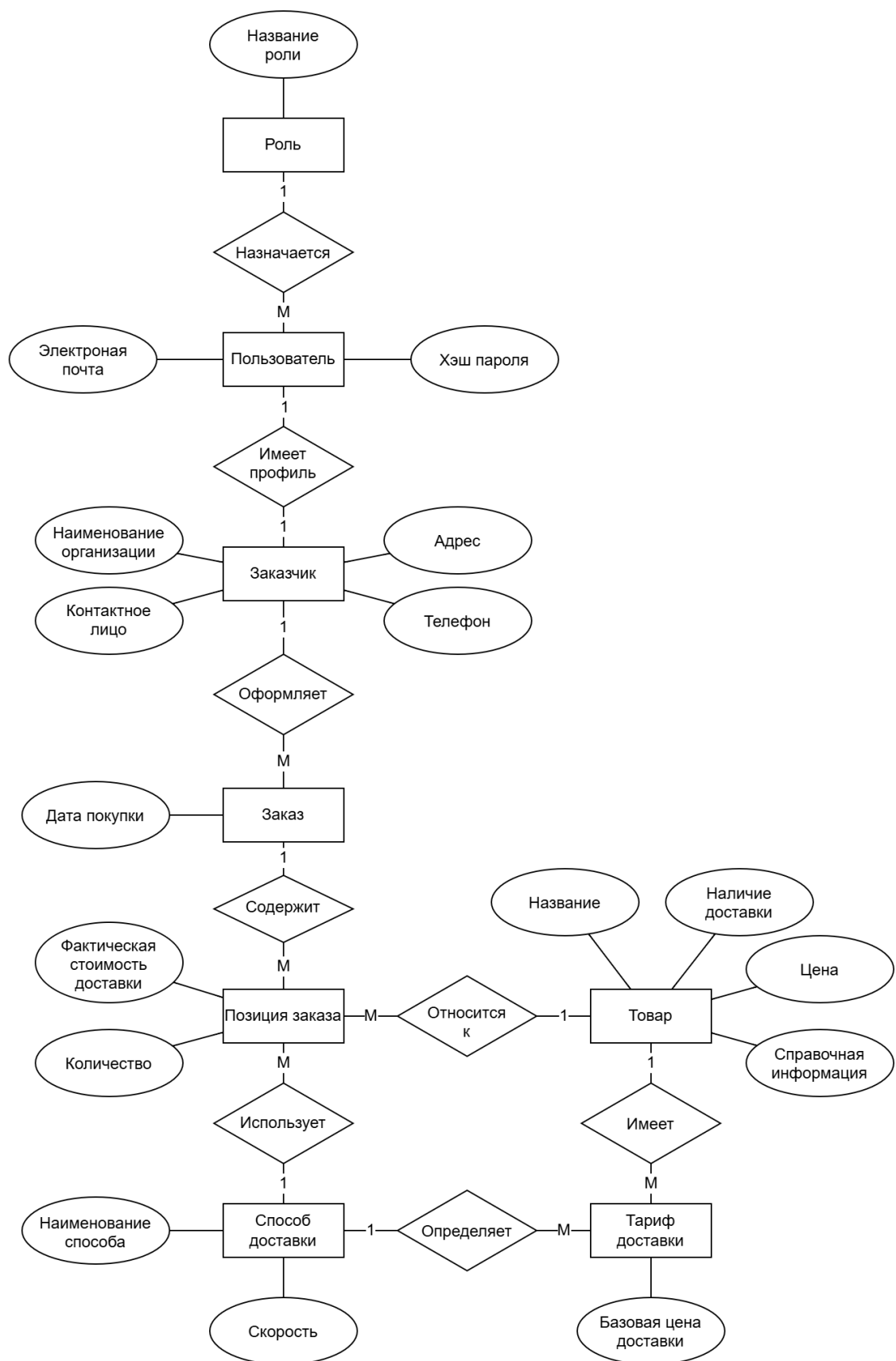


Рисунок 2 – Инфологическая модель базы данных (ER-диаграмма Чена)

2. Первичная реляционная модель данных представлена на рисунке 3. В ней все выявленные связи инфологической модели были прямо

транслированы в отдельные реляционные отношения (включая связующие таблицы «Имеют профиль», «Оформляют», «Содержат», «Относятся к», «Используют», «Определяют»).

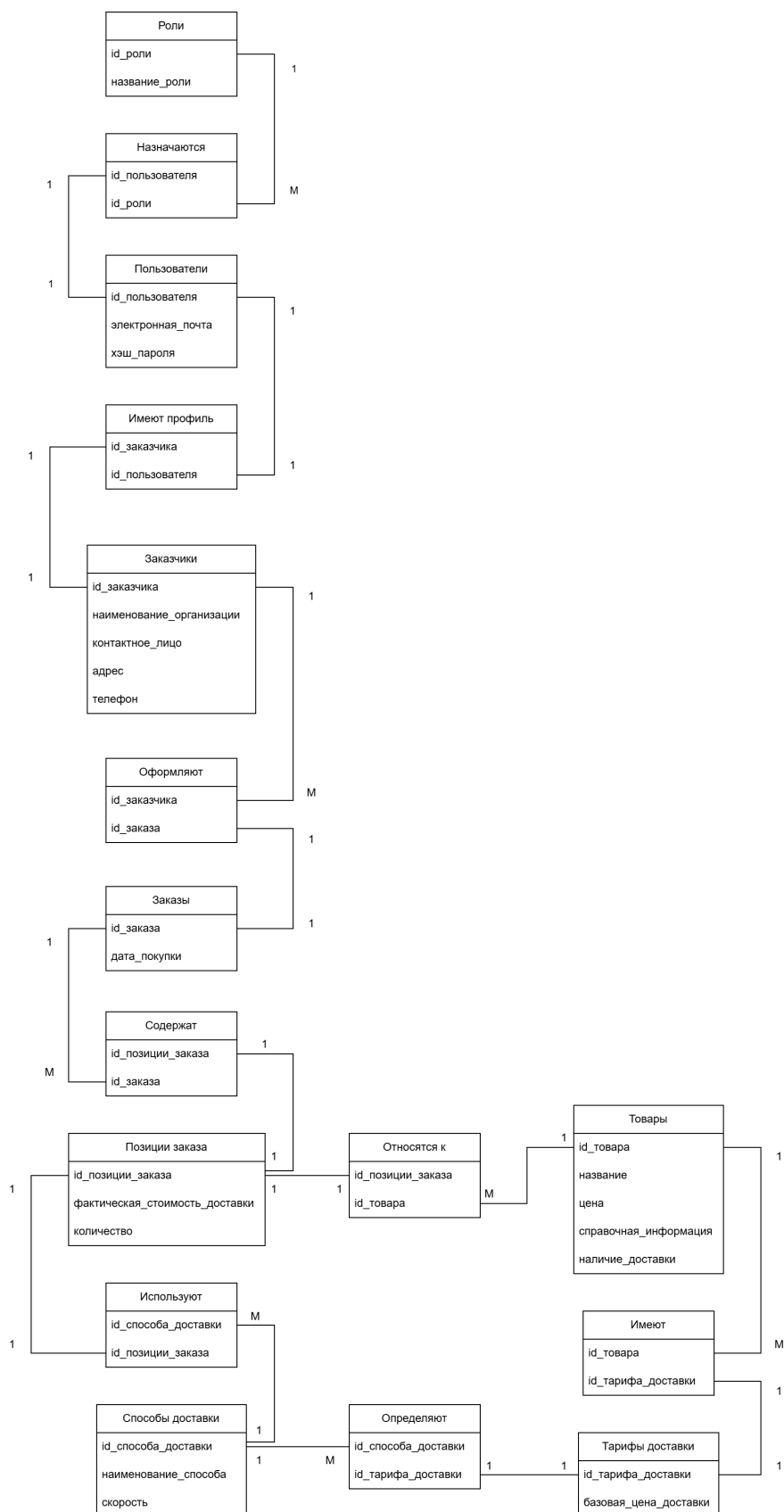


Рисунок 3 – Первичная реляционная модель базы данных

3. Итоговая оптимизированная реляционная модель данных представлена на рисунке 4. В результате реляционного соединения связей со связностью 1:1 («Пользователи» и «Заказчики») и оптимизации связей 1:M промежуточные таблицы были исключены с внедрением внешних ключей в дочерние отношения, что позволило сократить накладные расходы при выполнении SQL-запросов JOIN.

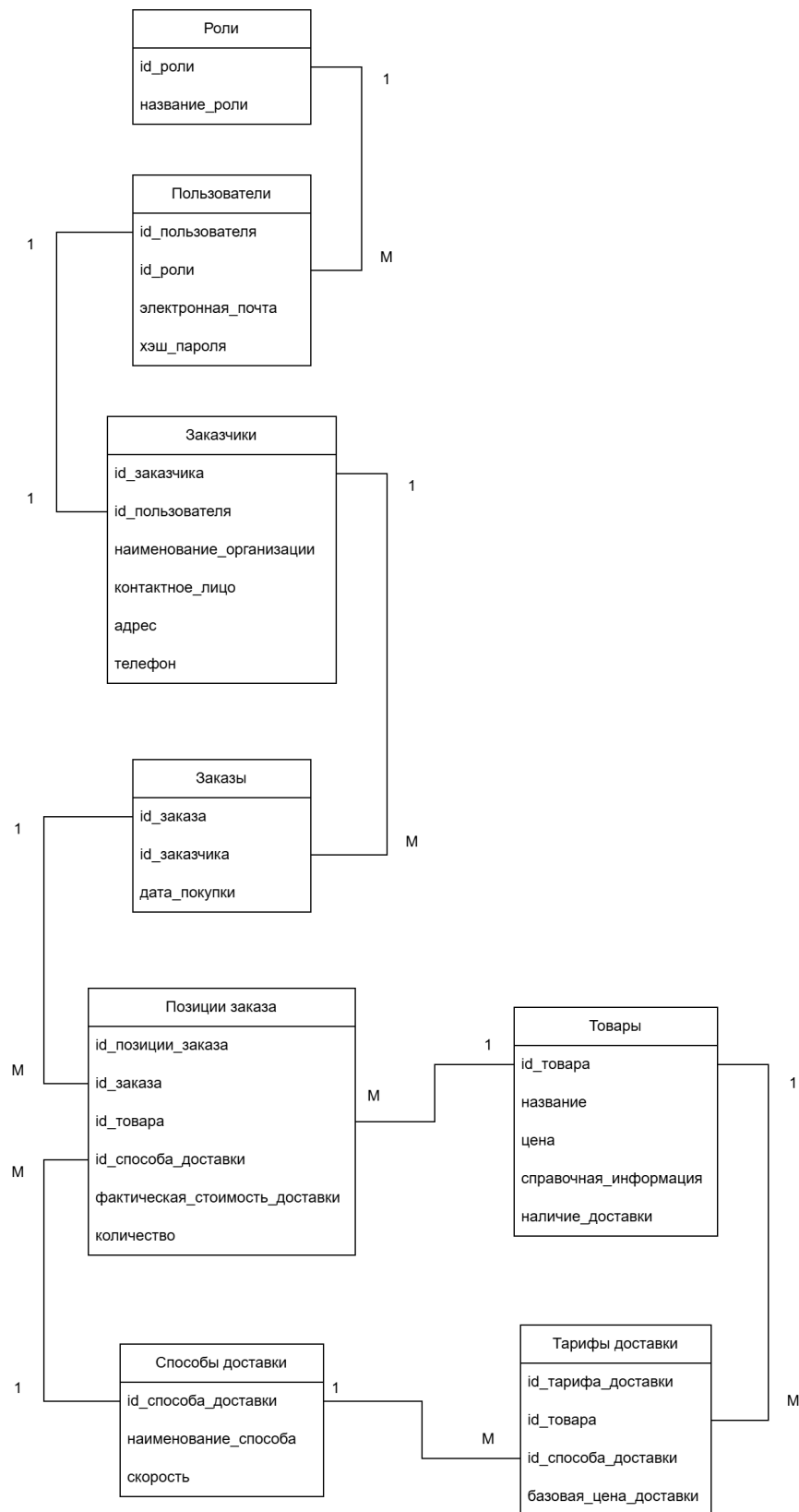


Рисунок 4 – Итоговая реляционная модель базы данных

4. Физическая модель базы данных, развернутая в СУБД PostgreSQL, представлена на рисунке 5. Схема сгенерирована средствами среды администрирования БД и наглядно отображает конкретные физические типы

данных СУБД (SERIAL, INT, VARCHAR, TEXT, NUMERIC(12,2), BOOLEAN, DATE), а также ограничения внешних ключей.



Рисунок 5 – Физическая модель базы данных в СУБД PostgreSQL

2.3 Структура кода

Архитектура программного комплекса основана на модульной клиент-серверной парадигме с распределением обязанностей между компонентами:

1. Сетевое взаимодействие и протокол: Обмен данными между клиентским приложением и сервером осуществляется по протоколу TCP через порт 12345 (класс Protocol). Сообщения передаются в формате легковесных пакетов JSON с разделителем строки '\n'. Запрос клиента содержит строковое действие action и словарь параметров params. Ответ сервера всегда содержит логический статус success (true/false), полезную нагрузку data или сообщение об ошибке error.

2. Серверная часть: Построена на базе библиотеки Qt6 Network. Класс `Server` наследуется от `QTcpServer` и переопределяет метод `incomingConnection`, создавая для каждого нового клиентского сокета индивидуальный объект-обработчик `ClientHandler`. Класс `ClientHandler` осуществляет разбор входящих пакетов, валидацию прав пользователя и вызов соответствующих методов уровня работы с данными.

3. Слой доступа к данным (`DatabaseManager`): Спроектирован с использованием классического порождающего паттерна проектирования «Одиночка» (`Singleton`). Конструктор и деструктор класса объявлены в приватной секции, операции копирования и присваивания удалены (`delete`). Доступ к единственному экземпляру осуществляется через потокобезопасный статический метод `instance()`. `DatabaseManager` инкапсулирует подключение к СУБД PostgreSQL (с возможностью прозрачного переключения на локальный файл SQLite `orders.db` при недоступности внешнего сервера), подготовку параметризованных SQL-запросов `QSqlQuery`, исключающих возможность SQL-инъекций, и преобразование наборов записей в объекты предметной области.

4. Клиентская часть: Представляет собой многооконное приложение на базе Qt6 Widgets. Сетевой обмен инкапсулирован в клиентском синглтоне `NetworkClient`, который поддерживает активное соединение, сохраняет токен и профиль текущей сессии (`currentUser`, `currentCustomer`) и предоставляет синхронный интерфейс `sendRequest`. Пользовательский интерфейс разбит на специализированные представления (`ProductView`, `OrderView`, `DeliveryView`, `CustomerView`, `StatsWidget`), объединенные в главном окне `MainWindow` с системой динамических вкладок и контекстных меню.

Взаимосвязь между компонентами клиентской и серверной частей, а также классами моделей предметной области представлена в виде UML-диаграммы классов на рисунке 6.

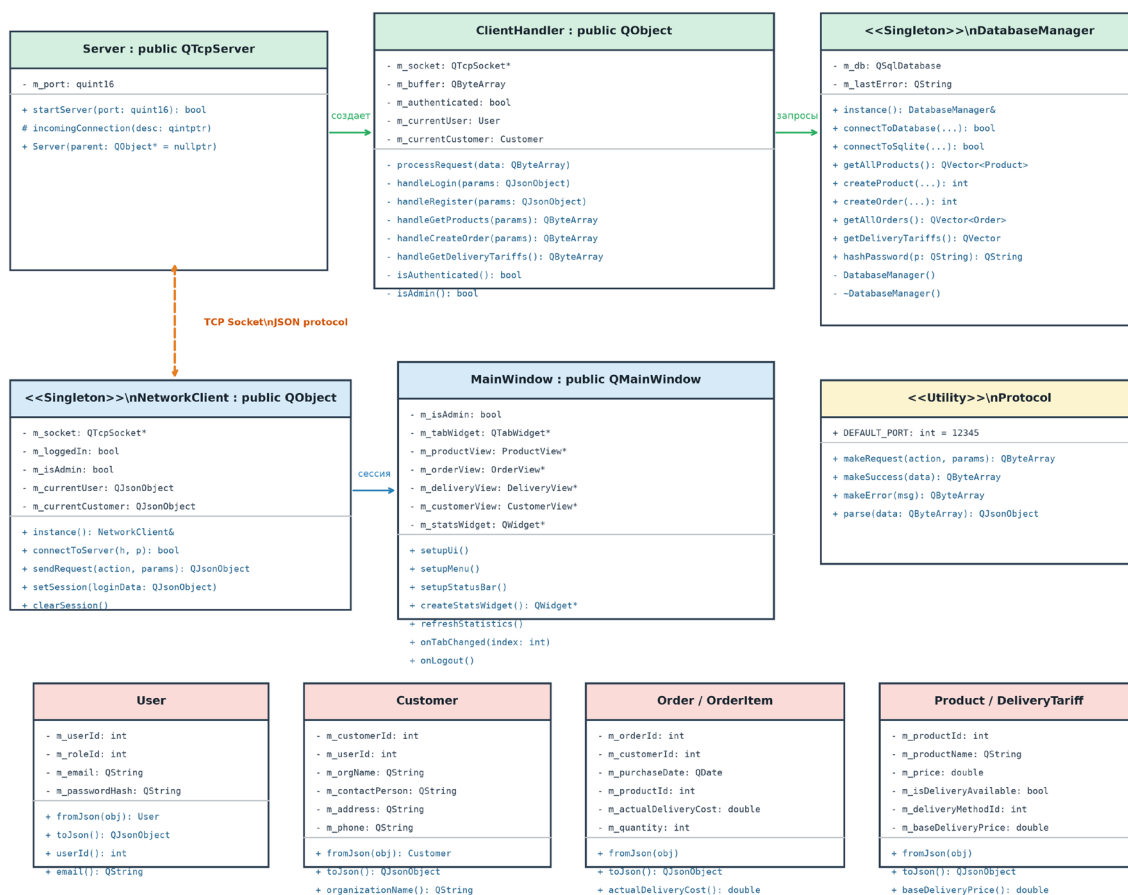


Рисунок 6 – UML-диаграмма классов клиент-серверной системы

Реализация класса DatabaseManager по паттерну «Singleton» проиллюстрирована на рисунке 7.

```

#ifndef DATABASEMANAGER_H
#define DATABASEMANAGER_H

#include <QString>
#include <QSqlDatabase>
#include <QSqlQuery>
#include <QCryptographicHash>

class DatabaseManager
{
public:
    // Потокобезопасная точка доступа к единственному экземпляру (Паттерн Singleton)
    static DatabaseManager& instance() {
        static DatabaseManager s_instance;
        return s_instance;
    }

    // Запрет копирования и присваивания
    DatabaseManager(const DatabaseManager&) = delete;
    DatabaseManager& operator=(const DatabaseManager&) = delete;

    // Подключение к СУБД (PostgreSQL и локальный fallback на SQLite)
    bool connectToDatabase(const QString &host, int port, const QString &dbName,
        const QString &user, const QString &password);
    bool connectToSqlite(const QString &dbFilePath = "orders.db");
    bool initializeSchema(const QString &sqlFilePath);

    // Бизнес-методы выполнения параметризованных SQL-запросов
    QVector<Product> getAllProducts();
    int createOrder(int customerId, const QString &purchaseDate);
    int createOrderItem(int orderId, int productId, int deliveryMethodId,
        double cost, int quantity);

    // Криптографическое хэширование паролей пользователей (SHA-256)
    static QString hashPassword(const QString &password);

private:
    // Приватные конструктор и деструктор гарантируют управление жизненным циклом
    DatabaseManager();
    ~DatabaseManager();

    QSqlDatabase m_db;
    QString m_lastError;
};

#endif // DATABASEMANAGER_H

```

Рисунок 7 – Заголовочный файл DatabaseManager.h по паттерну «Singleton»

3 ОСНОВНЫЕ СЦЕНАРИИ РАБОТЫ

3.1 Сценарий №1 – Регистрация пользователя и добавление данных о заказчике

Сценарий описывает процесс создания новой учетной записи контрагента:

1. Пользователь запускает клиентское приложение OrderClient. Открывается модальное окно авторизации LoginDialog. Пользователь переключается на вкладку «Регистрация».

2. Пользователь вводит адрес электронной почты (email), пароль и подтверждение пароля. В блоке «Данные организации» заполняются реквизиты юридического лица: официальное наименование организации (например, ООО "ТехноПромСнаб"), контактное лицо (Хакимов Дени Равилевич), фактический адрес и контактный номер телефона (+7 (495) 789-01-23).

3. При нажатии на кнопку «Зарегистрироваться» клиент проверяет совпадение паролей и отправляет серверу сетевой запрос action: 'register'.

4. Сервер проверяет сложность пароля регулярным выражением, вычисляет хэш SHA-256 и в рамках единой транзакции создает запись в таблице users с ролью customer, после чего связывает ее с новой записью в таблице customers.

5. При успешном ответе сервера выводится информационное сообщение «Регистрация прошла успешно! Теперь войдите в аккаунт». Пользователь переходит на вкладку «Вход», вводит свои учетные данные и попадает в главное окно системы MainWindow.

3.2 Сценарий №2 – Оформление заказа пользователем

Сценарий описывает выбор товаров и расчет стоимости с учетом predetermined вариантов доставки:

1. Авторизованный заказчик открывает вкладку «Заказы» главного окна. В верхней панели нажимает кнопку «Новый заказ». Система открывает диалоговое окно создания сделки, где фиксируется текущая дата покупки.

2. После создания заголовка заказа в таблице «Позиции заказа» становится активной кнопка «Добавить позицию». При ее нажатии открывается диалог добавления товара.

3. Заказчик выбирает наименование товара из выпадающего списка (например, «Ноутбук Pro 15» или «Монитор 27 4К»). Клиент отправляет фоновый запрос к серверу `get_delivery_tariffs` для выбранного товара.

4. В выпадающем списке способов доставки отображаются исключительно те варианты, которые предопределены для данного товара. При выборе способа (например, «Курьерская доставка (экспресс)» со скоростью 1-2 дня или «Самовывоз со склада») система автоматически подставляет соответствующий базовый тариф в поле стоимости доставки.

5. Заказчик указывает требуемое количество единиц товара (`quantity`). При подтверждении позиции сервер проверяет корректность параметров и фиксирует строку в таблице `order_items`. В таблице позиций моментально пересчитывается итоговая сумма строки: (Цена товара × Количество) + Фактическая стоимость доставки.

3.3 Сценарий №3 – Создание нового пользователя и управление справочниками администратором

Сценарий демонстрирует администрирование системы:

1. Администратор входит в систему под учетной записью с ролью `admin`. В главном окне активируются дополнительные вкладки и управляющие кнопки.

2. Управление номенклатурой: во вкладке «Товары» администратору доступны кнопки «Добавить», «Редактировать» и «Удалить». При добавлении открывается диалог `ProductEditDialog`, где вводятся наименование, цена, техническое описание и признак доступности доставки.

3. Управление тарифами: во вкладке «Доставка и тарифы» администратор настраивает доступные методы доставки (наименование, скорость) и формирует строки матрицы тарифов `delivery_tariffs`, жестко привязывающие допустимые способы к товарам и устанавливающие базовую расценку.

4. Управление заказчиками: во вкладке «Заказчики» администратор видит полный перечень организаций-контрагентов с реквизитами и телефонами, с возможностью оперативного аудита или удаления неактивных профилей.

3.4 Сценарий №4 – Просмотр истории всех заказов и финансовая аналитика

Сценарий отражает выполнение требований отслеживания финансовой стороны работы:

1. Администратор или ответственный менеджер переходит во вкладку «Финансовая статистика».

2. Клиентское приложение запрашивает массив всех проведенных сделок `get_orders` и позиций `get_order_items`.

3. В блоке «Финансовые показатели» автоматически рассчитываются и наглядно выводятся ключевые метрики предприятия: совокупное число заказов, суммарный физический объем проданных изделий (шт.), общая выручка от реализации продукции (руб.), затраты на логистику и транспортировку (руб.), а также средний чек заказа с учетом доставки.

4. В нижней табличной части формируется детализированный отчет «Продажи по товарам»: по каждому артикулу отображается базовая цена, суммарное количество отгруженных единиц и накопленный финансовый объем выручки, что позволяет оперативно оценивать маржинальность и динамику реализации различных групп товаров.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсового проекта по дисциплине «Проектирование и администрирование баз данных» была полностью спроектирована и программно реализована надежная клиент-серверная информационная система для ведения заказов торговой компанией (Вариант 4). Разработка выполнена на языке C++ с применением библиотек Qt6 и промышленной реляционной СУБД PostgreSQL.

Достоинства разработанного решения:

1. Надежная клиент-серверная сетевая архитектура. Разделение обязанностей между сервером баз данных, TCP-сервером бизнес-логики и клиентским приложением исключает прямое подключение рабочих станций к ядру СУБД, повышая отказоустойчивость и безопасность корпоративной сети.

2. Строгая математическая нормализация. Схема базы данных приведена к нормальной форме Бойса-Кодда (НФБК), что гарантирует отсутствие аномалий обновления, включения и удаления записей.

3. Точный учет логистических особенностей. Реализована гибкая система predetermined тарифов доставки: для каждого товара фиксируется перечень допустимых способов и базовых расценок, а в позиции заказа сохраняется историческая стоимость доставки на момент оформления сделки.

4. Криптографическая защита учетных записей. Внедрено хэширование паролей пользователей по криптостойкому алгоритму SHA-256 с валидацией сложности при регистрации.

5. Встроенный блок финансовой аналитики. Система в реальном времени формирует агрегированные экономические показатели (общая выручка, затраты на логистику, средний чек) и рейтинговую таблицу продаж по номенклатуре товаров.

6. Отказоустойчивость и автономность. Предусмотрен автоматический механизм перехода сервера на локальную базу данных SQLite в случае недоступности внешнего сервера PostgreSQL.

Ограничения учебного проекта:

- **Отсутствие сквозного TLS-шифрования:** В текущей версии сетевой протокол функционирует поверх открытого сокета TCP без включения протокола шифрования TLS/SSL, что допустимо для учебного стенда и изолированного сегмента сети, но требует сертификатов для открытого интернета.

- **Десктопный клиент:** Приложение ориентировано на десктопную платформу (Windows/Linux/macOS) и не предоставляет web-интерфейса для доступа из мобильных браузеров.

- **Отсутствие выделенного сервиса логирования:** Журналирование операций ведется в консольный поток сервера без записи в постоянную базу данных системных логов.

Перспективы развития проекта:

1. Интеграция протокола OpenSSL (QSslSocket) для обеспечения криптографической защиты сетевого трафика между клиентами и сервером.

2. Разработка web-версии клиентского кабинета на базе современных фреймворков и REST API.

3. Внедрение автоматизированной генерации счетов, накладных и товарно-транспортных документов в форматах PDF и Excel.

4. Интеграция с внешними сервисами отслеживания трек-номеров транспортных компаний.

Ссылка на репозиторий проекта: <https://github.com/z0nku7/kurs>

СПИСОК ЛИТЕРАТУРЫ И ИНТЕРНЕТ-РЕСУРСЫ

1. ГОСТ 19.106–78. Единая система программной документации. Требования к программным документам, выполненным печатным способом. – М.: Стандартиформ, 2010. – 8 с.
2. ГОСТ 19.404–79. Единая система программной документации. Пояснительная записка. Требования к содержанию и оформлению. – М.: Стандартиформ, 2010. – 4 с.
3. ГОСТ 7.32–2017. Межгосударственный стандарт. Отчет о научно-исследовательской работе. Структура и правила оформления. – М.: Стандартиформ, 2017. – 28 с.
4. Основная профессиональная образовательная программа по направлению подготовки 09.03.01 «Информатика и вычислительная техника», направленность (профиль) «Системная и программная инженерия». – М.: Московский политехнический университет, 2025. – Режим доступа: <https://mospolytech.ru/> (дата обращения: 11.09.2026).
5. Дейт, К. Дж. Введение в системы баз данных: пер. с англ. / К. Дж. Дейт. – 8-е изд. – М.: Издательский дом «Вильямс», 2018. – 1328 с.
6. Коннолли, Т. Базы данных. Проектирование, реализация и сопровождение. Теория и практика: пер. с англ. / Т. Коннолли, К. Бегг. – 3-е изд. – М.: Издательский дом «Вильямс», 2017. – 1440 с.
7. Шкрыль, А. А. Разработка клиент-серверных приложений в Qt / А. А. Шкрыль. – СПб.: БХВ-Петербург, 2020. – 480 с.
8. PostgreSQL Documentation. The PostgreSQL Global Development Group. [Электронный ресурс]. – Режим доступа: <https://www.postgresql.org/docs/current/> (дата обращения: 11.09.2026).
9. Qt Documentation. Qt Project. [Электронный ресурс]. – Режим доступа: <https://doc.qt.io/qt-6/> (дата обращения: 11.09.2026).
10. SQLite Documentation and SQL Database Engine. [Электронный ресурс]. – Режим доступа: <https://www.sqlite.org/docs.html> (дата обращения: 11.09.2026).

ПРИЛОЖЕНИЕ

Приложение 1 – Физическая модель базы данных (DDL SQL)

```
-- =====
-- Информационная система для ведения заказов компаний
-- Физическая модель БД (PostgreSQL)
-- Вариант 4. Хакимов Д.Р., группа 251-951
-- =====

-- 1. Роли (Roles)
CREATE TABLE IF NOT EXISTS roles (
    role_id SERIAL PRIMARY KEY,
    role_name VARCHAR(100) NOT NULL,
    CONSTRAINT uq_roles_role_name UNIQUE (role_name)
);

-- 2. Пользователи (Users)
CREATE TABLE IF NOT EXISTS users (
    user_id SERIAL PRIMARY KEY,
    role_id INT NOT NULL REFERENCES roles(role_id) ON DELETE RESTRICT ON UPDATE
    CASCADE,
    email VARCHAR(255) NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    CONSTRAINT uq_users_email UNIQUE (email)
);

-- 3. Заказчики (Customers) - связь 1:1 с пользователем
CREATE TABLE IF NOT EXISTS customers (
    customer_id SERIAL PRIMARY KEY,
    user_id INT NOT NULL REFERENCES users(user_id) ON DELETE CASCADE ON UPDATE
    CASCADE,
    organization_name VARCHAR(255) NOT NULL,
    contact_person VARCHAR(255) NOT NULL,
    address TEXT NOT NULL,
    phone VARCHAR(20) NOT NULL,
    CONSTRAINT uq_customers_user UNIQUE (user_id),
    CONSTRAINT uq_customers_phone UNIQUE (phone)
);

-- 4. Заказы (Orders)
CREATE TABLE IF NOT EXISTS orders (
    order_id SERIAL PRIMARY KEY,
    customer_id INT NOT NULL REFERENCES customers(customer_id) ON DELETE CASCADE ON
    UPDATE CASCADE,
    purchase_date DATE NOT NULL DEFAULT CURRENT_DATE
);

-- 5. Товары (Products)
CREATE TABLE IF NOT EXISTS products (
    product_id SERIAL PRIMARY KEY,
    product_name VARCHAR(255) NOT NULL,
    price NUMERIC(12, 2) NOT NULL CHECK (price >= 0),
    reference_info TEXT,
    is_delivery_available BOOLEAN NOT NULL DEFAULT TRUE
);

-- 6. Способы доставки (Delivery Methods)
CREATE TABLE IF NOT EXISTS delivery_methods (
    delivery_method_id SERIAL PRIMARY KEY,
    method_name VARCHAR(150) NOT NULL,
```

```

        speed VARCHAR(100) NOT NULL,
        CONSTRAINT uq_delivery_methods_name UNIQUE (method_name)
    );

-- 7. Тарифы доставки (Delivery Tariffs - связь M:M)
CREATE TABLE IF NOT EXISTS delivery_tariffs (
    delivery_tariff_id SERIAL PRIMARY KEY,
    product_id INT NOT NULL REFERENCES products(product_id) ON DELETE CASCADE ON
UPDATE CASCADE,
    delivery_method_id INT NOT NULL REFERENCES delivery_methods(delivery_method_id)
ON DELETE CASCADE ON UPDATE CASCADE,
    base_delivery_price NUMERIC(12, 2) NOT NULL CHECK (base_delivery_price >= 0),
    CONSTRAINT uq_delivery_tariffs_product_method UNIQUE (product_id,
delivery_method_id)
);

-- 8. Позиции заказа (Order Items - сохранение истории)
CREATE TABLE IF NOT EXISTS order_items (
    order_item_id SERIAL PRIMARY KEY,
    order_id INT NOT NULL REFERENCES orders(order_id) ON DELETE CASCADE ON UPDATE
CASCADE,
    product_id INT NOT NULL REFERENCES products(product_id) ON DELETE RESTRICT ON
UPDATE CASCADE,
    delivery_method_id INT NOT NULL REFERENCES delivery_methods(delivery_method_id)
ON DELETE RESTRICT ON UPDATE CASCADE,
    actual_delivery_cost NUMERIC(12, 2) NOT NULL CHECK (actual_delivery_cost >= 0),
    quantity INT NOT NULL CHECK (quantity > 0)
);

```

Приложение 2 – Реализация класса «Singleton» и сетевого протокола

```
// =====
// Фрагмент реализации протокола Protocol.h и синглтона DatabaseManager
// =====

// Protocol.h
namespace Protocol {
constexpr int DEFAULT_PORT = 12345;

inline QByteArray makeRequest(const QString &action, const QJsonObject &params = {})
{
    QJsonObject obj;
    obj["action"] = action;
    obj["params"] = params;
    return QJsonDocument(obj).toJson(QJsonDocument::Compact) + "\n";
}

inline QByteArray makeSuccess(const QJsonValue &data = QJsonValue()) {
    QJsonObject obj;
    obj["success"] = true;
    if (!data.isNull() && !data.isUndefined())
        obj["data"] = data;
    return QJsonDocument(obj).toJson(QJsonDocument::Compact) + "\n";
}

inline QByteArray makeError(const QString &message) {
    QJsonObject obj;
    obj["success"] = false;
    obj["error"] = message;
    return QJsonDocument(obj).toJson(QJsonDocument::Compact) + "\n";
}
} // namespace Protocol

// DatabaseManager.cpp (паттерн Singleton и хэширование SHA-256)
DatabaseManager& DatabaseManager::instance() {
    static DatabaseManager s_instance;
    return s_instance;
}

QString DatabaseManager::hashPassword(const QString &password) {
    QByteArray hash = QCryptographicHash::hash(password.toUtf8(),
    QCryptographicHash::Sha256);
    return QString::fromLatin1(hash.toHex());
}

int DatabaseManager::createOrder(int customerId, const QString &purchaseDate) {
    QSqlQuery q(m_db);
    q.prepare("INSERT INTO orders (customer_id, purchase_date) VALUES (:cid,
    :pdate)");
    q.bindValue(":cid", customerId);
    q.bindValue(":pdate", purchaseDate);
    if (!q.exec()) {
        m_lastError = q.lastError().text();
        return -1;
    }
    return q.lastInsertId().toInt();
}
```

Приложение 3 – Демонстрация графического интерфейса пользователя

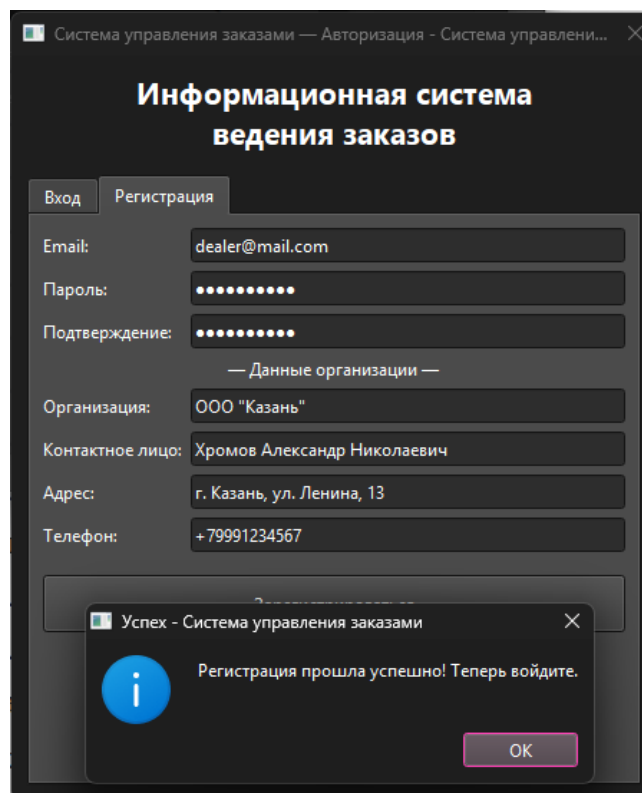


Рисунок П3.1 – Окно авторизации и регистрации учетной записи

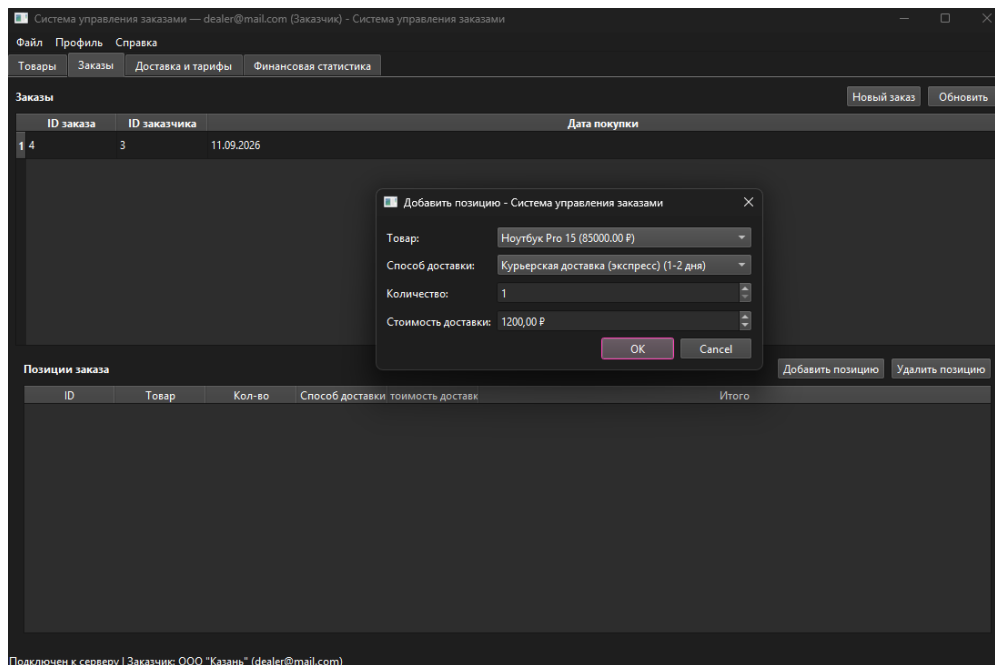


Рисунок П3.3 – Формирование заказа и выбор predetermined способа доставки

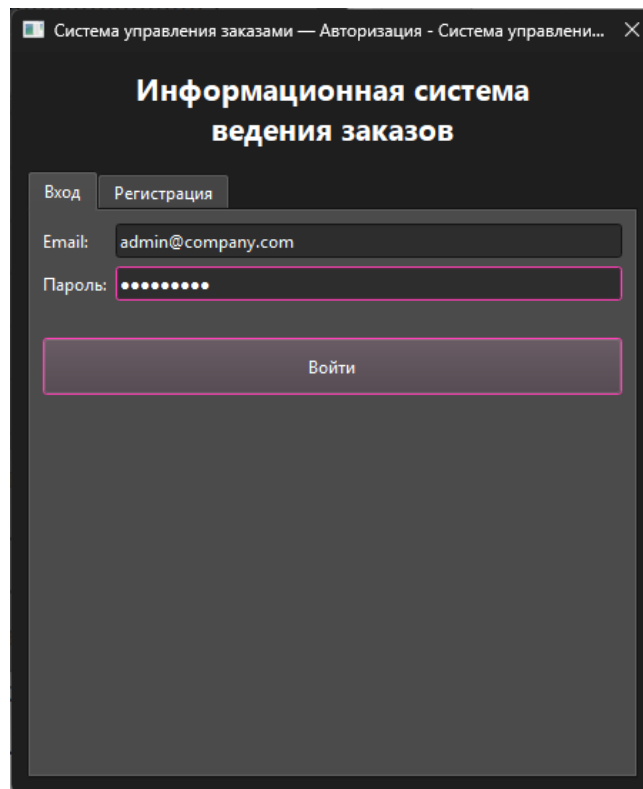


Рисунок ПЗ.5 – Вход в панель администратора

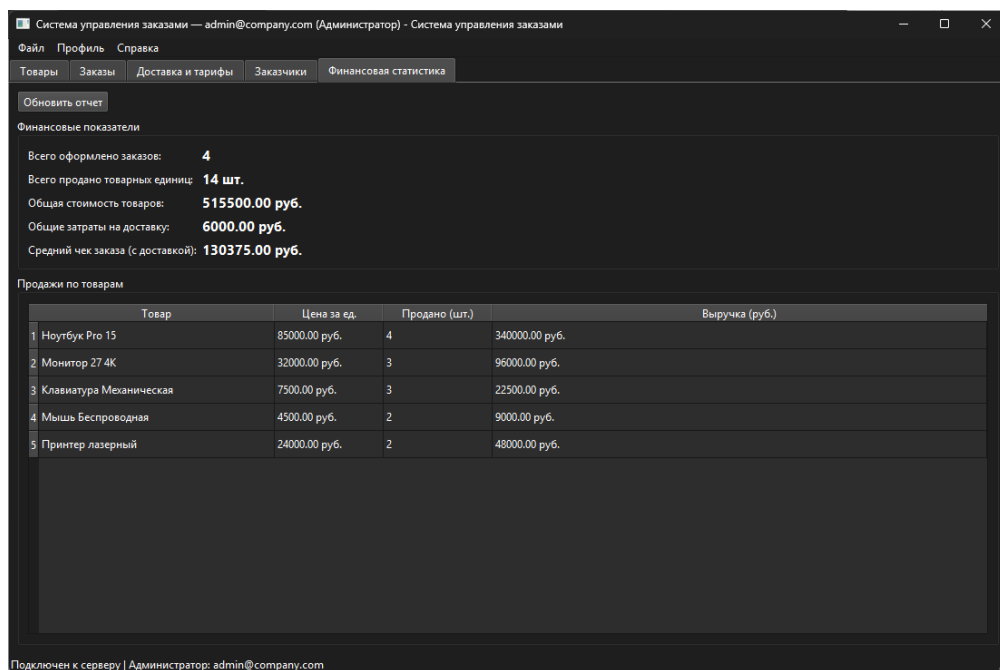


Рисунок ПЗ.6 – Просмотр журнала заказов и товарных позиций в панели управления